Multi-Agent Systems (BSc)
MARBEL

Koen Hindriks, Mike Ligthart
Course e-mail

Assignment 3: Single Agent BW4T

Available: November 27, 2025
Virtual deadline:
Hard deadline: December 15, 2025 (via Canvas, before **23:59PM**)

1 Introduction

There are many tasks that involve search and retrieval. Some examples that have gained much attention in the literature are *search and rescue*, *surveillance*, and *package delivery*. In the search and rescue domain, agents typically need to search for victims, and prioritise transport of victims that are badly hurt to an aid location and only after handling these victims provide aid to victims that were not in direct need of support. In the surveillance domain, agents continuously monitor a given area for intruders. In the package delivery domain, agents need to collect and transport packages from a pickup location to a destination location. These tasks differ on various dimensions. For example, a map of the area to be covered is typically given in surveillance and package delivery tasks but may not be available in a search and rescue mission; the location of items to be retrieved is given in a package delivery task, but items must be searched for in the search and rescue and surveillance tasks.

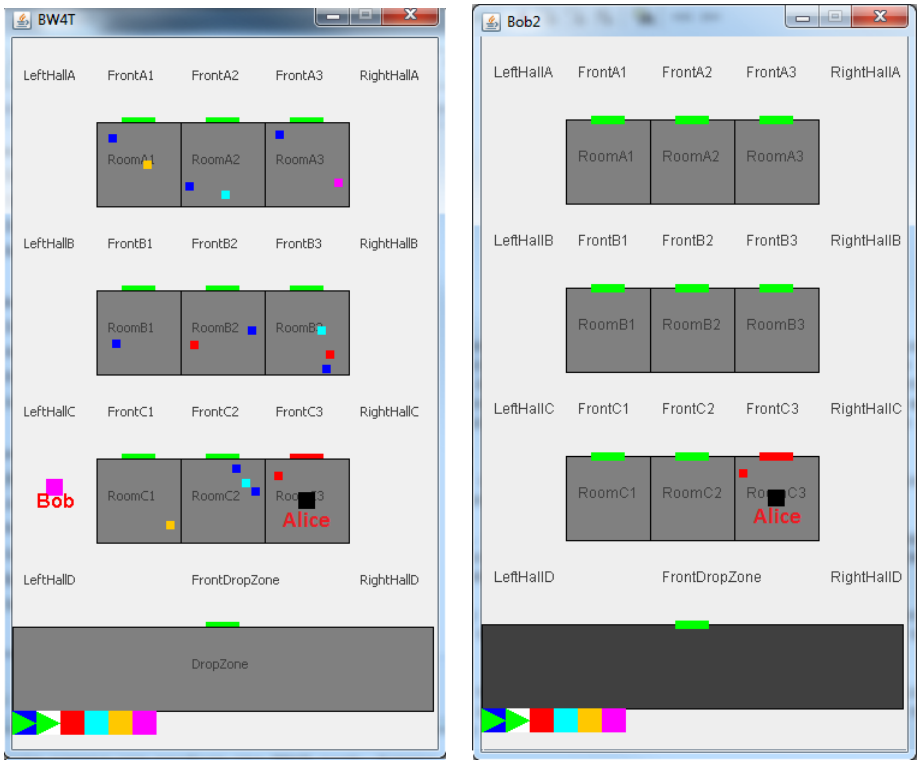
In order to design good strategies for executing search and retrieval tasks, we do not necessarily need to model all the complex details of these various domains. In this assignment, we use the *Blocks World for Teams* (BW4T) environment as an abstract representation for search and retrieval tasks such as search and rescue. BW4T is a virtual world that consists of *rooms* in which *colored blocks* may or may not be located, and a so-called *dropzone* where blocks can be delivered. The BW4T task is to deliver a sequence of colored blocks in a particular order. Performance on the BW4T task is measured by the speed of completing the task (time-to-complete). The BW4T task can be performed by a single agent or by multiple agents. This assignment sheet is about developing a single MARBEL agent that can solve the BW4T task.

2 The Blocks World for Teams Environment

Figure 1 shows two screenshots of the BW4T environment. The left picture shows all blocks and all robots in the environment. The blocks are located in nine rooms (room A1-C3), and there are two robots in this session: Alice and Bob. Bob is in the left hall and is holding a pink block. Alice is in room C3 and is not holding any block. The right picture shows what robot Alice can see at this point in time. Alice cannot see Bob, and because Alice is in room C3, she can only see the red block in this room.

To deliver a block successfully, a robot has to go to a block of the right color, pick it up, go to the dropzone at the bottom of the environment and drop the block in the dropzone. A robot can only carry one block at a time. The status bar below the dropzone shows which blocks need to be delivered and which blocks already have been successfully delivered. In this example, a dark blue and a white block have already been delivered (indicated by the green triangles), and a red, light blue, orange, and pink block still need to be delivered. All robots can keep track of the status bar because they receive a percept with information about the current state of the status bar. So when one robot delivers a block of the right color, all robots are informed that that color has been delivered. When a robot drops a block of the wrong color in the dropzone or when it drops any block in a hall, the block disappears from the environment. A block that is dropped in an arbitrary room does not disappear from the environment. Two robots cannot be in a room at the same time. In general, robots cannot see each other, but when a robot enters an occupied room, a red bar indicating that the room is occupied appears, which all robots can see as well because they receive a percept with information about occupied rooms.

Developing a robot that can perform the BW4T task on its own requires that robot to explore, locate, and pickup the blocks needed and deliver them in the right order to the dropzone. For this to happen, the robot needs to decide which rooms to check and which blocks that it finds to deliver.



(a) Complete information view

(b) Incomplete information view

Figure 1: Fully observable view (left) and limited agent perspective (right) on BW4T

Summarizing, the robot that you will program for this assignment should be a rule-based agent program that is able to complete the BW4T task by searching and delivering blocks of the right color in the right order.

3 Assignment Objectives and Instructions

This assignment must be completed *in pairs*, i.e. two students work together on a solution. This section briefly summarizes the objectives, what you need to get started, the requirements, the deliverables and how you should submit your solution. Section 4 describes the assignment itself in detail. Please make sure to read this assignment sheet carefully before starting to work on your agent program!

3.1 Objective

To learn to design and write an agent program that creates an agent that processes percepts, represents knowledge and maintains beliefs about the BW4T environment, is able to decide which action to take next based on its goals, and performs these actions to successfully complete the BW4T task.

3.2 What Do You Need to Get Started?

For this assignment, you will need to use the MARBEL plug-in for Eclipse. We assume that you have already installed Eclipse and this plug-in. In addition, you will need to install the BW4T environment and server. If you have not already done so, please follow the step-by-step instructions below to do this.

1. In Eclipse, if you have not done so yet, please select the MARBEL perspective (Window → Perspective → Open Perspective → Other → MARBEL).
2. Load the BW4T project in Eclipse (File → New → Project... → MARBEL Agent Programming → MARBEL Example Project → BW4T). This project consists of a set of MARBEL **files** and the **BW4T client** JAR.
3. The project that you just loaded also contains two pdf files with environment documentation: one pdf with a **specification** of the BW4T environment that you will need to complete this assignment, and one pdf with additional **instructions**.
4. Download the **BW4T server**, i.e., the environment the agent operates in. Do so by visiting this link and download the latest release of the BW4T server `jar` file in a folder of your choice.

See the assignment below for instructions on how to start the server and launch the initial MAS file provided to you in the BW4T project.

3.3 Deliverables

The deliverable for this assignment is:

- A single **zip file** that contains your solution for this programming assignment, including all relevant MARBEL **source code** files. Do **NOT** include any BW4T documentation files, JAR files, etcetera.

Other file formats, e.g. 7-Zip, screenshots of code, etc. will not be graded!

You need to submit the zip file on Canvas using the Assignment 3 submission page that will be made available.

3.4 Minimal Requirements

You will use the MARBEL agent programming language to program an agent that controls the BW4T robot's choice of actions. Your solution should meet the following minimal requirements:

- Your `mas2g` file contains your *contact information*, i.e., your names, student numbers, and email addresses are included in comments at the beginning of your `mas2g` file.
- Your code *compiles and runs*, i.e., the MARBEL source files parse correctly, compile, and launch an agent that controls and moves a robot in the BW4T environment.
- Your code does *not depend on 'hard coding'*, e.g., it does not contain any room identifiers with the exception of the `'DropZone'`.
- Your code files all contain *explanatory comments* that will allow third parties not involved in programming the code to understand it. Please make sure you incorporate enough information in comments in the code to facilitate understanding. **VERY IMPORTANT: Without sufficient comments, the assignment will be marked insufficient.** Please double-check whether your comments make your code sufficiently clear by asking someone else to have a look before you hand in (e.g. a TA).
- Your agent should be able to deliver all blocks in the goal sequence in order *on any map that comes with the BW4T environment*.

4 Assignment: Single Agent BW4T

This assignment consists of two parts:

1. The first part consists of *questions* about the BW4T environment. You do NOT need to submit the answers to these questions, but should try to study them in order to gain a better understanding of the MARBEL language and the BW4T environment.
2. The second part consists of the *design and adaptation* of the agent program initially provided such that it is able to perform the BW4T task. For this, you will need to improve the performance of this single agent program in the BW4T environment.

4.1 Part I: Understanding MARBEL and the BW4T

As a start, let's get the BW4T environment and a MAS for controlling a robot in that environment up and running. The BW4T environment runs on a separate server with which MARBEL connects. Therefore, the server should be started *before* the MARBEL agent can connect to the environment.

- Start the server by running the BW4T server `jar` file, either by double-clicking it (if you do so in Eclipse, ignore/cancel any popups) or using the `java -jar` command in a terminal window (make sure you have installed Java version 11). A window with the BW4T environment as in Figure 1 should appear.
- Run the `.mas2g` file. To do this in Eclipse, select the *Debug As* menu item after right-clicking on the file (or use the debug button in the menu).[1] In Eclipse an agent called *robot* should be launched and in the BW4T interface a robot should appear that starts moving in the environment.

To restart the MARBEL agent and the BW4T environment after the agent has been killed, always first **press the *Reset* button in the environment before restarting the MAS file in Eclipse**.

Exercise 1 (Understanding MARBEL , 10 points)**.** To help you get started, we ask you to first try to understand the agent program that comes with the project. In order to gain a better understanding of the behavior of the agent that is initially provided in the BW4T project, inspect the agent program code and observe the behavior of this agent in the environment. You should try to answer the following questions for yourself, **but you do NOT need to include the answers in your submission!**

1. Run the agent and restart the agent a couple of times and observe its behavior. Which sequence of rooms did the robot visit each time? Can the robot visit the same room twice?
2. Does the agent produce the same behavior every time or not? Why is the agent's behavior (non-)deterministic? To answer this question, it is useful to pause the agent several times and to evaluate the query `not_yet_visited(P).` in the interactive console. You can also use a (conditional) breakpoint to make the agent pause at a rule when that rule is applicable.
3. Inspect the agent code. What do you think the agent is doing? Do not only look at the observable behavior of the agent in the environment, but also the internal actions used for updating the agent's state.
4. The `state/1` predicate in the agent program plays an important role in determining when the goTo action can be executed. Inspect how this predicate is updated by creating and searching

[1] For more information, please see the User Manual at this link. Note that you can also quickly run agents by using the *Run As* menu item (or the corresponding menu button), which does not switch the Eclipse perspective to the MARBEL debug mode.

for the predicate in a log file that logs all percepts.[2] Where in the agent program is this predicate used?[3] Remove the part of the rule which uses the predicate and run the `.mas2g` file again. What happens?

5. Can you run the agent program without problems on an arbitrary map (in the BW4T environment window, use the menu at the top to change maps)? Why would it or would it not run without problems?

6. (a) The BW4T environment can be viewed as an abstract representation of tasks such as search and rescue. That is, various aspects present in the BW4T environment (rooms, robots, colored blocks, dropzone, etc.) relate to aspects of search and retrieval tasks. Can you map the relevant features of a search and rescue task onto a BW4T environment? (Use the BW4T Specification pdf that you can find in the project folder in Eclipse.)

   (b) Where would the mapping fail? Or, alternatively, why do you think the BW4T provides an adequate representation of search and retrieval tasks?

4.2 Part II: Programming in MARBEL

The objective of the second part of the assignment is to enable the agent to deliver all blocks in the goal sequence. For that, the agent needs to be modified such that:

- it processes the relevant percepts it receives from the environment.
- it reasons about which color needs to be delivered.
- it selects and performs the appropriate actions to achieve its main goal.

Below, we will go through these modifications step-by-step.

Before we begin, please note that the convention is to use **camelCase for writing BW4T actions**. That is, you need to write e.g. `goTo` instead of `goto`.

Exercise 2 (BW4T, 90 points)**.**

1. **Processing percepts.** (15 points) The BW4T sends messages with information about the environment to agents that are connected to it. Our agent will need, for example, the information about the goal to be achieved and the state of the BW4T to be able to complete its task.

   Currently, the agent only processes information regarding places (where what is and where the agent has been). The *BW4T Specification document* that comes with the project provides information on percepts that the environment can send to the agent, and specifies their type.

(a) **Add relevant percepts to the `mas2g` file.** (5 points) Read the BW4T Specification document and identify which percepts you should add to the agent definition in the `.mas2g` file. You should add exactly 5 percepts that the agent really needs for task completion. *Hint:*

- The agent needs to know the goal it needs to achieve and the progress it has made. The goal is a sequence of colored blocks and progress is indicated by an index into that sequence. Obviously the agent needs to also know the color of blocks.
- The agent needs to pick up blocks. To do so, it needs to be able to verify the preconditions of the `pickUp(<BlockID>)` action so check these out. Robots can only hold one block at a time.

[2] In the Eclipse menu, select Window → Preferences → MARBEL → Logging, check 'Write logs to files' and 'Percepts processed'. Make a note where you can find the logging directory and Apply and Close. Then run your agent again to create the log file. You can also inspect the agent tab in the console area in Eclipse but the output length is limited while a log file stores the complete history of the agent's execution.

[3] In Eclipse, you can use the Search menu to search for `state(` to locate all occurrences in an enclosing project (select this option while making sure you selected a project file or are editing one).

Use the type of the percept to decide which of the percept operators `add`, `replace`, or `update` should be used. Also add explanatory comments on what information the percepts provide!

To see the effects of this change, run the program in *debug mode* and switch to the *debug perspective*. See the User Manual on how to do this. When the agent is killed, inspect the database of the agent. Which information has been added to the database?

Check whether the colors in the BW4T window match with the sequence list of colors in the database.

It is informative to run the agent again in debug mode and pause the agent when it is in a room (select the MARBEL Debugging Engine and press pause). What additional information do you see in the database?

(b) **Declaring predicates so they can be used.** (3 points) The percept predicates are added to the database of the agent automatically. However, to be able to use them in the agent program for querying and updating the database, we need to explicitly declare them. To do so, open the `bw4tKnowledge.pl` file and add the new percept predicates to the list of declarations.

In Eclipse, check the Problems console. You should see now five new warnings that say a predicate is defined but never used. This indicates that a predicate may have been redundantly declared, but in our case it indicates that we are ready to use the predicates in our program!

(c) **Update the agent's database.** (7 points) As you will have noticed, the color information is removed again when a robot leaves a room again. It only receives ('sees') that information when it is in a room. Our agent, therefore, should store the information *that it has seen a block of a particular color in a room* before it leaves the room again.

To implement this, declare a new predicate `block(BlockID, ColorID, PlaceID)` which we can use to combine this information in the agent's database. Now add a rule of the form `forall ... do ...` to the `robotUpdate.mod2g` file to insert `block/3` facts into the database by combining all the relevant information in the rule's query (block id, color, and place).

(d) **Test.** Run the agent again in debug mode and wait until the agent has been terminated again when it arrives in the dropzone. Then inspect the agent's database.[4] Do you see information about blocks stored in the agent's database? Does this information match with the blocks that are actually present in the rooms in the environment? If not, you need to check your code!

2. **Changing the condition for terminating the agent.** (5 points) Before we continue, we should change the condition for terminating the agent. Clearly, doing this when the agent has visited all rooms does not guarantee that the agent completed its task.

   In order to fix this, replace the query `not( not_yet_visited(_) )` in the `robotControl.mod2g` file with a query that (only) succeeds when the task has been completed. You will need to inspect the length of the goal sequence using the `sequence(List)` fact and compare it with `Index` given by the `sequenceIndex(Index)` fact.

   Hint. The built-in Prolog predicate `length/2` will be useful.[5]

3. **Implementing the agent's decision logic.** (30 points) Now that the agent is able to acquire the right information about the environment (where to find blocks of the right color that it needs), we are going to implement the logic that will enable the agent to make a decision about what to do next at each turn.

   Informally, the agent needs to deliver the next block that is first in the sequence of colored blocks that still need to be delivered. To achieve that, the following recipe needs to implemented: (1) go to a room in which the agent can find a block of the right color, (2) go to that block and (3) pick it up, then (4) move to the dropzone, and, finally, (5) drop the block it holds there.

   The agent should repeat this recipe until the overall goal has been achieved.

[4] It is useful to right-click on a fact in the database view and select Find. If you then type `block(` you will only see the `block/3` facts listed.

[5] You can find a list of the SWI Prolog predicates that are supported by MARBEL here.

In MARBEL , the strategy for implementing such a recipe is to program a set of rules for each of these steps and prioritize the most important rule to apply next. That is, the rule that should be applied next (is most important) should be put at the top of the list of rules, then the second-most important thing to do, etc.

(a) **Prolog rule for figuring out which color needs to be delivered next.** (5 points) Define a predicate named `next_color_needed(ColorId)` in the `bw4tKnowledge.pl` file by using the predicates `sequence/1` and `sequenceIndex/1`.

The idea is to retrieve the `Index`*th* element, given by the `sequenceIndex(Index)` fact, in the list of colors that need to be delivered, given by the `sequence(List)` fact. *Hint.* The built-in Prolog predicate `nth0/3` will be useful.

(b) **Decide to go to a room with a block of the right color.** (5 points) The next thing to do is make the agent go to a room where it can find a block with a color that matches the next color in the goal sequence that needs to be delivered. Using the Prolog rule you just defined, add such a rule to the `main` module, i.e. the `robotControl.mod2g` file.

Hint. For the rule's condition, use `next_color_needed(ColorId)` to retrieve the color needed, `block(BlockID, ColorID, PlaceID)` to retrieve a room where the agent can find a block of that color, and, finally, do not forget to check that the agent is not already going somewhere (inspect the rule that is already in the main module for how to do that).

(c) **Test.** Run the agent in debug mode and check out what happens (don't forget to reset the environment). Vary the location where you place the new rule in the `robotControl.mod2g` file (put it at the top, put it as second rule, and as last rule). Does the agent get stuck in a room that has a block with a color that is needed next before it has visited all the rooms?

(d) **A greedy strategy.** We will implement a *greedy* strategy to complete the task. *To this end, make sure you put the rule for going to a room with a block of the color that is needed next at the top of the file.* In the remainder of this assignment subtask 3 where you implement the agent's decision logic, keep adding each new rule that you write to the top of the `robotControl.mod2g` file.

(e) **Going to a block to pick it up.** (5 points) Let's assume that the robot is in a room with a block that it can use next (i.e. of a color that is needed next to complete the task). The robot can only pick up that block if it is within reach distance of the block (check the BW4T Specification documentation). Our next task thus is to make sure that is the case.

Write a new rule to make the robot move closer towards a block *if it is not yet within reach*. Of course, the robot should only try to do that if it is already in a room with the block it wants to pick up! Also always check using the `state/1` predicate that the robot is not already doing something. Add the rule to the `robotControl.mod2g` file.

(f) **Picking up one block at a time.** (5 points) Our next task is to pick up the block, assuming that the robot is now standing next to a block that is needed. We will write a program where the robot will pick one block at a time (in line with its gripper capacity).

Write a rule for picking up a block and add the rule to the `robotControl.mod2g` file. As before, we need to check our assumptions and add those to the query of the rule we need to write. Also check that the robot is not holding a block already!

(g) **Test.** Run the agent in debug mode and wait until the agent gets stuck. Pause the agent and check that it is holding a block by inspecting its database.

(h) **Delivering a block to the dropzone.** (5 points) Our next task is to make the robot deliver a block to the dropzone. We will assume that if the robot is holding a block already, then it only still needs to carry it to the dropzone.

Write a rule for going to the dropzone and add the rule to the `robotControl.mod2g` file. As before, we need to check our assumptions. That is, we need to query that the robot is holding a block in the rule we need to write. Do we need to check for anything else in the query of the rule?

(i) **Dropping a block in the dropzone.** (5 points) Finally, assuming that the robot will only go to the dropzone if it is holding a block, the robot should drop the block it is holding whenever it is in the dropzone.

Write a rule for dropping a block the robot is holding when it is in de dropzone and add it to the main module.

(j) **Test.** Run the agent in debug mode and check if the robot is delivering blocks to the dropzone.

4. **Completing the task.** (25 points) The agent program we have written so far is delivering blocks of the right color to the dropzone. But will the robot always complete the task?

(a) **Fixing the problem.** (15 points) The robot sometimes gets stuck before the task is completed. Identify the problem by debugging the agent. Simply wait until the agent gets stuck somewhere and pause the agent. By inspecting the database, identify why the robot is not moving to a block it needs next.

Add a rule to the `robotUpdate.mod2g` file that fixes the problem.

(b) **Test your solution.** (10 points) Your agent should be able to deliver all blocks in the goal sequence in order *on any map that comes with the BW4T environment*. Make sure your agent works on these maps by testing, for example, whether it is able to complete the BW4T task on the *Banana* map. Moreover, your agent should terminate itself when it has completed its task. Also test to see if that happens.

5. **Changing the logic. Being less greedy.** (10 points) An alternative strategy to complete the task is to first finish exploring the environment until all the blocks that are needed have been located, and only then start delivering the blocks to the dropzone.

Implement this strategy by defining a Prolog rule for a predicate `found_all_blocks_still_needed/0`. Besides the main Prolog rule defining this predicate, you will want to define additional clauses for helper predicates to write the rule for this predicate; more specifically, you will want to check that one list is a sublist of the other. *Hint.* It will be useful to look into the Prolog built-in predicates `findall/3`, `lenght/2`, and `append/3`.

After defining the `found_all_blocks_still_needed/0` predicate, add it to the queries of some of the rules in the `robotControl.mod2g` file. To which rules should you add the predicate to make sure the robot is no longer behaving too greedy?

Hint. It will be useful to evaluate your code by using conditional breakpoints, the database inspector, and the query console in the debug mode perspective. See the User Manual for more on how to do that.

6. **Combining conditions by using nested rules.** (5 points) Finally, note that several rules share queries. For example, the first and second rule should both have the `holding(_)` query as part of those rule's conditions. Restructure your rules (but do not re-order them!) and rewrite them so that such duplicate queries can be removed as much as possible. You should be able to group rules at least 3 times.